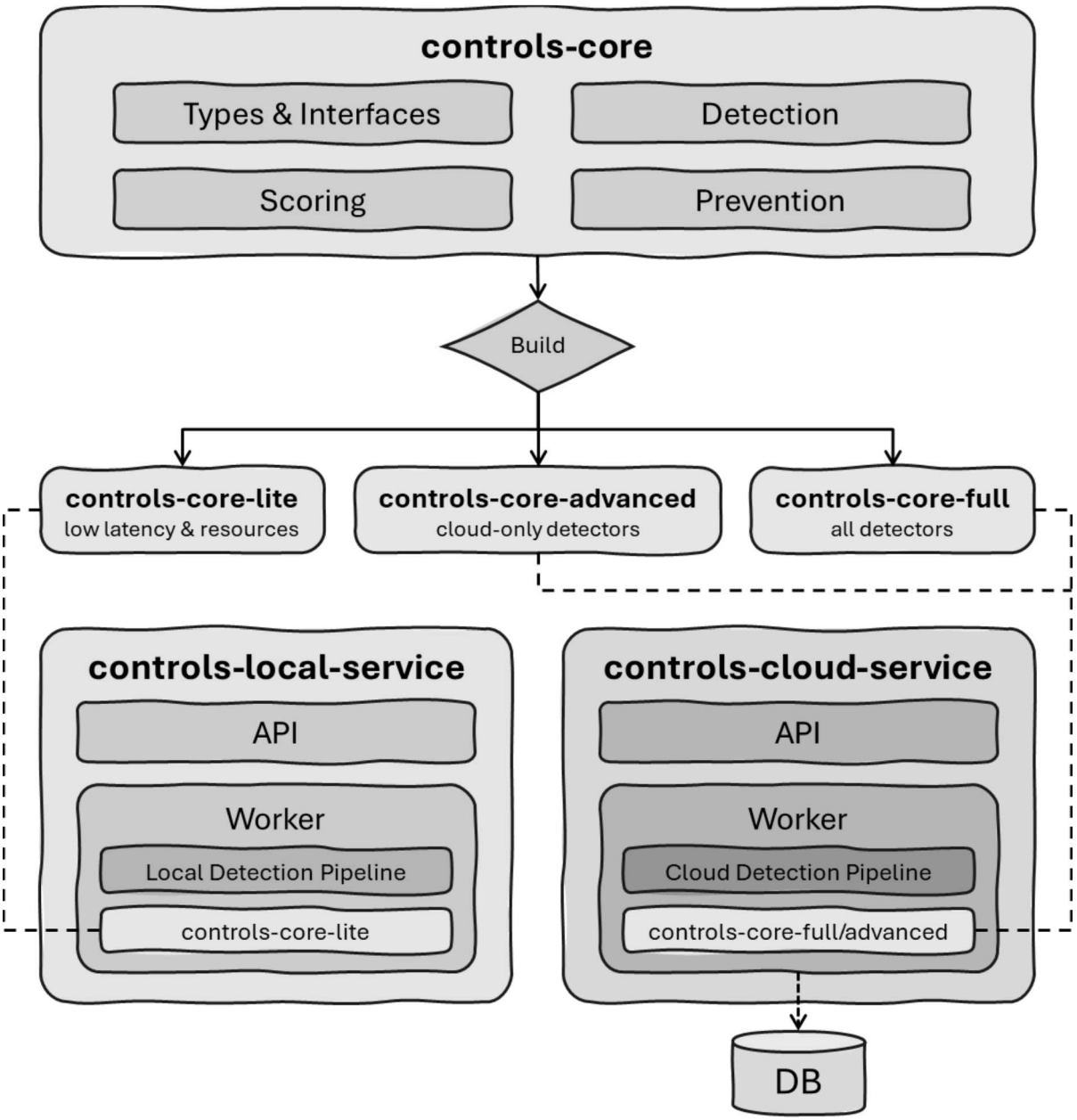
AI Controls | Design & Consideration

由 Shaked Hindi创建, 最后修改于4月 15, 2026 57 views since 6 Apr 2026

Architecture Design



General

	Local service	Cloud service
Logic	Core	Full or Advanced (depending on

		deployment type)
Logging	Local log + rotation	PostgreSQL (api key management, logging and post-analysis)
API framework	fastapi + uvicorn	Gateway + stateless workers deployment for horizontal scaling
Storage	All local	Shared NFS/Object storage (for resources/models)
Noteable Features	• Resource limiter	• API Key Management • Dataset scanning

Accepted Inputs

	Definition
Prompt	Prompt is the input being sent by the user to the AI model
Response	Response is the output being produced by the AI model, in response for a given prompt
Dataset	Dataset is a set of prompts and their corresponding responses (either expected or actual), to be examined in batch by the controls.

Execution Strategy

Detectors are grouped into priority tiers by estimated latency.
Within each tier, eligible detectors can run in parallel.
After each tier, a cumulative score is evaluated.
If **"fail-fast" is enabled** and the score exceeds the threshold, remaining tiers are skipped.

	Tier	**Detector Types**
Core	**0**	Language detection (if not supplied)
	1	Exact matches + compiled regex
	2	Structural heuristics (entropy, char ratio, repetition, patterns)

	Tier	Detector Types
	3	"Classic" ML models and small ONNX models
Advanced	4	Advanced heuristics (heavy-lifting)
	5	Relatively-fast SLM & LLM models
	6	LLM-as-Judge (thinking & reasonsing, only for borderline use-cases)

Possible Optimizations

	Logic
Language Detection	Always runs first, eliminates most of regex patterns for non-matching languages
Multi-keyword Matching	Use **pyahocorasick** fast and efficient search of multiple keywords in a single pass
Fail-fast	After some/every tier: if cumulative score over threshold, skip all remaining tiers
Advanced pipeline	Only invoked if core score is in uncertain range (e.g., 0.25–0.80)
LLM-as-Judge	Only invoked if score is in borderline range (e.g., 0.35–0.70) after tier 5
Category filtering	allow to skip categories and focus only on categories of interest

Extension Strategy

Adding a new detector, category, or language should require **zero changes** to orchestration, aggregation, or API code. The system discovers and loads extensions automatically.

Adding a New Detector

1. Create Files and resources

```
detection/
├── ...
└── my_detector/
    ├── __init__.py
    ├── detector.py                    # Detector logic
```

```
    ├── detector.toml                    # Detector config & metadata
    └── resources/                       # Detector resources
        ├── example1.txt
        └── example2.txt
```

2. Implement the logic

```
class MyDetector(Detector):
    detector_id = "my_detector"
    category    = "my_category"
    priority    = 2
    resource_tier = "core"
    execution_tier = 2
    ...
```

3. Add to registry

```
@DetectorRegistry.register
class MyDetector(Detector):
    ...
```

4. Done! The registry auto-discovers all registered detectors via Python package import scanning at startup.
   Note: Detectors can be **enabled/disabled** per deployment via config without code changes.

Consideration & Open Questions

1. **unavailability** - we should define how customers should act when we crash or too busy
2. **cloud deployment**
   a. gateway - which to use?
   b. rate limiting?
   c. shared storage - nfs / object storage?
3. **local deployement**
   a. security - defend against theft, bypass, corruption, etc.
   b. logs - should be sent to cloud db upon acceptance?
   c. local db - is it required?
   d. config - should it exist? should we sign it with an external key?
4. **datasets**
   a. Better understand how datasets look like and how they should be scanned properly
   b. what size should we support?
   c. async API?
   d. push/pull?
5. **detectors**
   a. Define and prioritize which controls should be implemented for each usecase (input / output / dataset)
   b. streaming output detectors - sliding window? sentence based? other?
   c. Strictly define the evaluation and resource testing protocol - where, how, etc.
   d. MoE - do we need it? If so how can we integrate it?
6. **update mechanism**

Follow up discussions

Topic	Info	Tasks
Logging	1. logging in local deployments a. logging for pre-deployment • input & output • which detectors b. logging for deployment • annonymized metadata • qira specific 2. logging for cloud deployments a. qira specific	
ML training pipeline	Decide on how it should be implemented	Prepare baseline for discussion
Security in local deployments		
Optimized detector internal decision making with decision points	• executor with list of tuples • threshold per detection capbility • how should we accumulate threshold on the go (phrase, phrase + pattern, etc.)	

无标签